Ripple: Testing What Language Models See and Decide in Robot Fleets

Ethan Moosbrugger Boos
Georgia Institute of Technology

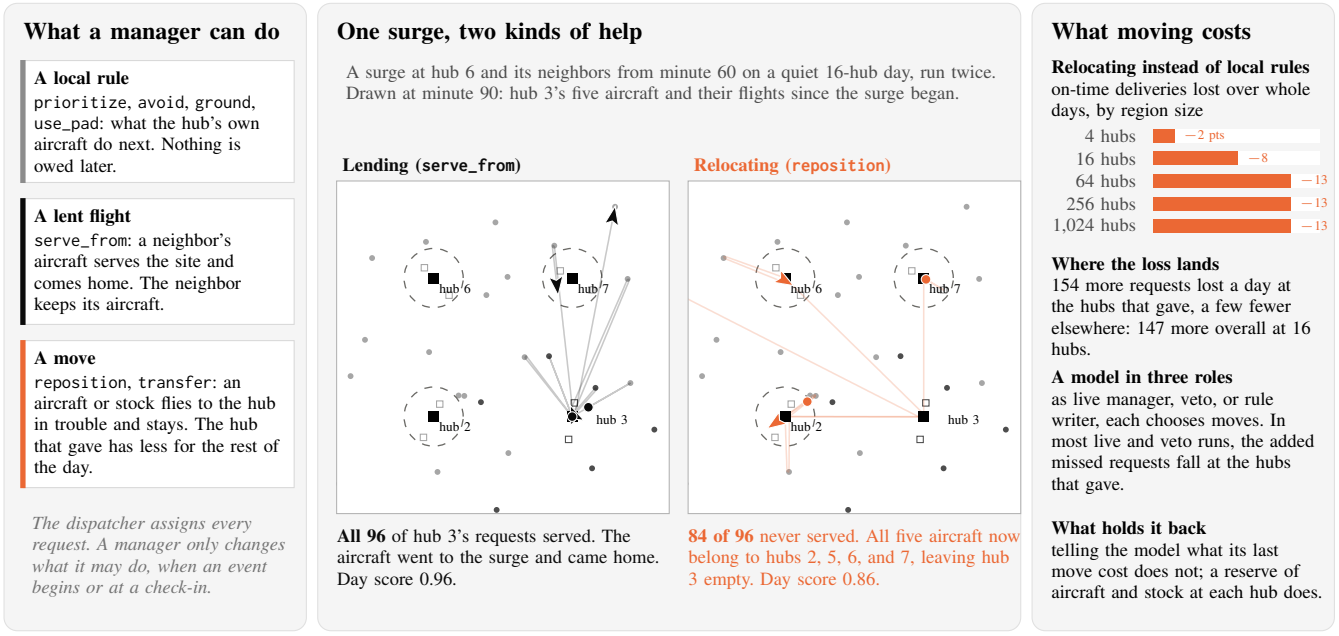


Fig. 1. Help at one hub can cost the hub that gave. Left: local rules, a lent flight, and a move of aircraft or stock. Middle: the same 16-hub surge (dashed rings) under lending and relocating. Hub 3's five aircraft (fixed-wing arrowheads, multirotor dots) fly to the surge and return under lending; under relocating, all five leave for hubs 2, 5, 6, and 7. Its clinics (dots) and hospital rooftops (squares) get no flight for the rest of the day, though both policies serve the surge about equally well. Right: fixed-policy cost by region size, added missed requests by hub role, and model managers' moves. All results are from simulation.

***Abstract*—Language models are increasingly proposed as managers for robot fleets, issuing orders that vehicles execute. An order may address an urgent request while leaving the fleet less able to serve later ones. We present Ripple, a multi-agent simulator for studying fleet managers over complete missions. Ripple lets us vary managers, dispatchers, vehicle behaviors, controllers, and motion models on matched missions. Without model calls, one worker runs an 8.5-hour day with 1,280 aircraft in 16 minutes and with 5,120 in about two and a half hours. In a regional delivery study, managers set local rules, allow a neighboring hub to serve requests, or move aircraft and stock. Language-model managers over-help: they relocate aircraft and stock when local rules and lending serve more requests on time. At 64 hubs, even a fixed relocation policy scores 13 points below local rules, with the added misses mainly at hubs that gave. Relocation can help when surges last longer. We also test a fleet API for choosing which hub should lend aircraft. That choice changes deliveries, but a simple rule based on spare aircraft matches Haiku and Opus at 16, 64, and 256 hubs without model calls. All results are from simulation.**

I. Introduction

Robot systems increasingly divide a longer task into what to do next and how to execute it. $\pi_{0.5}$ predicts a semantic subtask, such as picking up a plate, before generating actions with the same model [1]. GR00T 2 lets a separate reasoning agent choose steps and monitor a robot policy as it carries them out [2]. These systems now handle long, multi-stage tasks on individual robots. A fleet planner must also decide which robot should act, how work should be shared, and what help now costs later. Language models are attractive for this role because an operator can describe new tasks or disruptions in ordinary language. Their orders pass through dispatchers, vehicle autonomy, and controllers; the effects unfold across the whole fleet.

Language-model planning for teams has taken several forms. SMART-LLM breaks an instruction into tasks for groups of robots [3]; COHERENT revises assignments to a mixed team after feedback [4]. Other systems compare centralized and local planners [5], direct large collectives from language [6], or dispatch and reposition a fleet of vehicles [7]. These systems show ways to connect a model to a team of robots. We study two questions about a language model managing a fleet through a full day. First, when requests and disruptions compete for the same aircraft and stock, how does the manager respond, and where do its decisions make the fleet

worse off? Second, what does a manager need to see to make a fleet decision, and does a model improve on a simple rule as the fleet grows? This second question concerns the fleet API: the report given to the model and the changes it may order.

We study these questions with a regional delivery fleet. Each depot, which we call a hub, has its own aircraft and stock. Requests arrive throughout the day, and a dispatcher assigns them to available vehicles. When urgent requests surge across several hubs, a manager can change which requests take priority, allow an aircraft from a neighboring hub to make a delivery and return home, or move aircraft and stock to the affected hub. Each choice may help with the surge. Moving aircraft and stock also leaves the neighboring hub with less capacity when its next requests arrive. The manager must decide whether the immediate help is worth that later cost.

Judging a plan only when the surge begins hides this later cost. The same aircraft may serve another request after returning home, or it may still be based elsewhere. Its route and return time determine which later requests it can reach before their deadlines. We therefore compare managers on the same stream of requests and disruptions, and score what the vehicles flew and delivered over the complete mission. Two runs with the same requests can still diverge if a model gives different replies. Each order changes the fleet state seen at later decisions.

We built Ripple, a multi-agent simulator, to run these comparisons. A manager gives orders to a dispatcher; vehicle autonomies, controllers, and motion models carry them out on the same clock. Metrics follow the flights and requests after each order, including work outside the immediate disruption. We can change the manager, its view of the fleet, or the vehicle model while running matched missions. Model replies are saved so each observed mission can be inspected and replayed. The fixed-policy study runs the same 8.5-hour mission with up to 5,120 aircraft. That scale lets us measure how a change to one hub affects work elsewhere while requests keep arriving across the region.

II. Related Work

Planning for robot teams. Language models have been used to decompose tasks and assign them to teams of heterogeneous robots. SMART-LLM forms coalitions for subtasks [3]; COHERENT revises assignments after feedback [4]. Chen et al. compare centralized, decentralized, and hybrid organizations as team size grows [5]. Scale-Plan filters a larger action graph before calling a model [8]. COMPASS studies language-conditioned collective behavior with up to 1,024 robots in short flocking trials [6]. These systems ask how a team can carry out a command. We ask what happens after a fleet manager changes the distribution of vehicles and stock over a day, including at hubs it was not trying to help.

Managing a fleet. Dispatch and repositioning are coupled: a vehicle sent to one area is no longer available in another. LLM-ODDR uses a language model to dispatch orders and reposition drivers; its surge study uses 300 taxis [7]. In aerial robotics, COLA executes multi-UAV missions through collaborating agents [9], and Web-of-Drones connects a language agent to flight tools [10]. The delivery problem here has heterogeneous vehicles, hub stock, deadlines, and rules that change during a mission. We give the manager several ways to help, including temporary service by a neighbor, and measure their later cost in the same operating day. Fixed policies run with up to 5,120 aircraft. Live regional managers are tested with 80 and 320 aircraft; fleet API models also choose lenders for a region with 1,280 aircraft.

Interfaces and checks. The interface through which an agent observes and acts can change its behavior without changing the model. SWE-agent shows this for software agents [11]; work on tool documentation and context selection studies particular parts of that interface [12, 13]. In robotics, SayCan grounds skill selection in affordances [14], while Safety Chip and RoboGuard check proposed actions against constraints [15, 16]. LLM-Modulo argues for verifiers around a model rather than treating its output as an executable plan [17]. Our checker rejects infeasible or rule-breaking orders; a separate guard limits how much capacity can leave any one hub. We compare those mechanisms with a smaller view and with a model that reviews a replanner's orders.

III. Ripple

Ripple runs a fleet whose vehicles and manager can be changed independently. Its design follows SCRIMMAGE's separation of agents, plugins, and world interactions [18]. Each agent owns physical state and a stack of plugins. The engine advances every agent through the same ordered phases, so a fleet decision, a vehicle's response, and a later request belong to one mission history. A research crate supplies the agents and algorithms for a study; the engine supplies timing, messages, recording, and metrics. The delivery crate used here defines hubs, a dispatcher, couriers, manager orders, and the score. The same engine can run a different autonomy or motion model without changing the simulation loop.

A. Composing a fleet

An autonomy chooses what an agent does. A controller turns its commands into motion inputs, and a motion model advances the physical state. Sensors produce readings; estimators update the belief available to autonomy and controllers. Interactions, networks, and metrics operate across agents. The mission selects these components for each agent. In this study, fixed-wing aircraft and multirotors share the request stream but use different controllers and motion models. Fixed-wing aircraft make a parachute drop on a pass; multirotors can also land on hospital rooftops. A manager need only choose among actions the vehicle autonomies expose.

The delivery crate defines the vehicle behaviors for this study. Its Courier autonomy handles the vehicle's lifecycle—idle, serving a request, returning, reloading, relocating, or lost—and runs a delivery skill for each assignment. The skill library describes which aircraft can `drop` or `land` and supplies

the behavior at the site; an assignment can also include a waypoint around a closed area. The dispatcher checks availability, skill, and estimated endurance before assigning a request. A model can change fleet rules through manager orders, while the dispatcher still selects a feasible skill and the courier flies it. The score is based on that flight. A request that looks feasible from the manager's summary can still become late while an aircraft reaches it and returns for another.

B. From a manager order to a flight

The dispatcher assigns individual requests to vehicles. At each decision it considers idle vehicles at the site's hub, stock, skill, estimated range, and arrival time. A request it cannot assign is retried. This dispatcher runs in every condition, including the condition with no manager. The manager acts above it when an event begins or at a scheduled check-in. It can change local priorities and restrictions, permit another hub to serve a site, relocate aircraft, or transfer stock by flight. The manager never assigns an individual request.

An order changes the dispatcher's future choices as well as the immediate flight. `serve_from` lets a neighboring hub serve a site, after which the aircraft returns to its own hub. `reposition` flies an idle aircraft to a new hub, which becomes its home. `transfer` uses a vehicle to carry stock between hubs. The latter two consume aircraft time and leave the hub that gave them less capacity for later requests. This difference is central to the experiment: a move can solve the current shortage and still reduce the day's on-time share. Ripple records each order's acceptance and the subsequent flight, delivery, and hub state.

For the comparisons below, six fixed policies and three ways of using a language model issue orders to the same dispatcher and couriers. For a live model, the delivery crate writes a view of the fleet and active events, calls the model, parses its reply into the same orders a scripted policy can issue, and applies them. A model can instead review a replanner's proposed orders and drop some, or write a policy file once before the day. The latter runs without model calls during the mission. The couriers still execute the same skills, and the metric scores their flights by the same rule.

C. Running matched missions

Ripple's ordered phases put the manager and vehicles on the same timeline: autonomy acts, controllers and motion models execute, sensors and world interactions update, messages are routed, estimators update belief, and metrics observe the result. The manager's view is formed from reported fleet state and events rather than the physical truth used by the metric. A closure, for example, is judged from a vehicle's actual position during the closure window. A delivery whose flight broke an active rule does not count as served, even if the manager ordered it in time.

Requests and events are drawn from random numbers keyed by hub and event index. A seed therefore defines a common day for managers that give different orders. Randomness in the simulator is keyed by purpose and stable identities, so a run repeats exactly. We also save each model reply with the prompt that produced it. An unchanged model run can then be replayed without another call. These properties make it possible to inspect a failed mission and to compare interventions on matched request streams. A changed view or policy changes future prompts and may require new model replies; saved replies alone do not make different conditions identical.

The simulator carries the whole region between manager calls. In the largest fixed-policy runs, it advances 5,120 aircraft through 306,000 steps of 0.1 seconds while requests, stock, and events change across the region. Requests arrive for the first eight hours of that 8.5-hour day. A run with 1,280 aircraft and no model calls took about 16 minutes on one worker, roughly 32 times faster than real time; a run with 5,120 aircraft took two and a half hours of processor time on one worker, about 3.4 times faster than real time, at a peak of 1.5 GB of memory. Model inference is timed separately from the simulator.

IV. Experimental Evaluation

We first use Ripple to study fleet decisions on a day of regional medical delivery. Hubs lie on a square grid 15 km apart (Figure 1). Each hub has three fixed-wing aircraft, two multirotors, four blood-product stocks, six clinics, and two hospitals with rooftops and nearby ground pads. Delivery requests arrive at twelve per hub per hour, with more during surges. Routine, urgent, and trauma requests have two-hour, 30-minute, and 15-minute deadlines. Requests arrive for eight hours, and the fleet has 30 minutes to finish the last of them. We compare fixed policies at 4, 16, and 64 hubs, or 20 to 320 aircraft, and also run them at 256 and 1,024 hubs. The model comparisons for this day use 16 and 64 hubs. The fleet API study uses a separate mission described in Section V. Within each comparison, policies and models face matched requests and events; the robustness checks deliberately change the workload or fleet.

The scale is that of a real operation. Zipline's long-range aircraft cruises at 97 km/h and flies round trips of over 190 km [19]; our fixed-wing aircraft cruise at 90 km/h, and the 15 km hub spacing and 4 to 13 km site distances sit well inside that range.

Events include trauma surges, stock shortages, hub outages, closed airspace, weather restrictions, priority changes, and rooftop closures. Their counts are drawn per hub, so a larger region encounters more disruptions. The same seed gives the same requests and events to every manager, so each comparison is on matched days. The base fleet uses a kinematic fixed-wing aircraft and a point multirotor with limited speed and endurance. Ripple also has checked six-degree-of-freedom fixed-wing and multirotor models; the results here do not establish that the manager ranking persists under those models.

The outcome is the on-time share of requests, weighted one for routine, two for urgent, and four for trauma. We also report late and missed requests, lost vehicles, rule breaches, accepted and refused orders, moves of aircraft or stock, and lending

permissions. A *move* is an applied relocation or stock transfer; a *lend* is an applied permission for a neighboring hub to serve a site, not necessarily a completed flight. The score is computed from deliveries and trajectories, not from the manager's intent.

Six fixed policies, rules written once and applied the same way all day, set the bar. *No manager* is the dispatcher alone. *Local rules* adds orders that change only what a hub's own aircraft do: priorities, closures, grounding, and ground-pad delivery. *Lending* adds permission for a neighboring hub to serve a site, after which the aircraft comes home. *Relocating* adds aircraft moves and stock transfers to local rules, without lending. The *replanner* uses every order and also tops up low stock at check-ins. *Return* is the replanner with its relocated aircraft sent home once the event ends. The policies give the same orders through the same dispatcher and are compared on the same days.

A language model takes three roles. As the *live manager* it is called when an event begins and every 30 minutes, reads a view of the fleet, and gives orders. As a *veto* it reads the replanner's proposed orders and drops the ones it rejects, but adds none. As the *author of the rules* it writes a policy file once, before the day, which then runs with no further calls. Four models, Claude Haiku, Sonnet, and Opus and GPT-6 Luna, are compared in the first two roles, on three seeds at 16 hubs and two at 64. Luna uses the OpenAI API; the Claude models use the Claude command-line client with tools disabled. Both paths receive the same study system prompt, with no tools available. The comparisons of what the model sees and the policy writing use Haiku and Sonnet. Every model call used one frozen system prompt and one text per event kind; the prompt, the texts, one complete call and one veto with their replies, and the per-run and per-seed numbers are in the appendix, in the repository.[1]

We ask how a language-model manager handles competing needs across a fleet and where its choices fail. To understand those choices, we first measure the cost of relocating aircraft and stock under fixed policies, with no model involved. We then test models as live managers, vetoes, and authors of rules: what they order, where the resulting missed requests fall, and what limits harmful moves. Finally, we examine what a model should see and be allowed to order as the fleet grows. We compare models with fixed rules for choosing a lender and change the information in their reports.

A. What moving costs, with no model

Table I compares the fixed policies, with no model involved. The score is the weighted share of requests delivered on time. At four hubs, lending scores 0.81 and the replanner ties local rules at 0.78. At 16 hubs, local rules and lending score 0.74, the replanner 0.69, and relocating 0.66. The gap between local rules and relocating reaches 13 points at 64 hubs and stays there at 256 and 1,024. From 64 hubs on, the replanner does worse than no manager. It grants the same lending permissions as the lending policy, but also relocates aircraft and transfers stock. Sending relocated aircraft home restores about half its gap to local rules at those sizes. Across all 20 days at 16 and 64 hubs, local rules beat both the replanner and relocating.

[1] https://github.com/EthanMBoos/ripple/tree/main/paper/appendix

TABLE I
MEAN WEIGHTED ON-TIME SCORE BY REGION SIZE. TEN SEEDS AT 4–64 HUBS; THREE AT 256 AND 1,024. BOLD MARKS ROW MAXIMA AT DISPLAYED PRECISION.

	No relocation			Relocation allowed		
Hubs	No manager	Local rules	Lending	Relocating	Replanner	Return
4	0.73	0.78	**0.81**	0.77	0.78	0.79
16	0.68	**0.74**	**0.74**	0.66	0.69	0.72
64	0.67	**0.73**	**0.73**	0.60	0.62	0.67
256	0.65	**0.71**	**0.71**	0.58	0.61	0.66
1,024	0.65	**0.71**	**0.71**	0.58	0.61	0.65

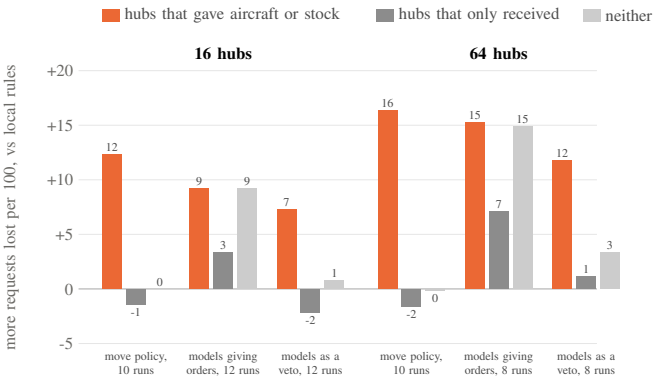


Fig. 2. Where additional missed requests fall. Hubs are grouped by whether they gave aircraft or stock, only received, or neither. Bars show additional late or undelivered requests per hundred compared with Local rules on the same seeded day. Results are pooled across the labeled number of runs; the live-manager and veto groups each combine Haiku, Sonnet, Opus, and Luna. Positive values indicate worse performance than Local rules. **TODO: replace Luna's original CLI runs in this figure with the API reruns.** For the relocating policy and vetoes, losses concentrate at donor hubs; live-manager losses are more widely distributed (Section IV-B).

The result survives two changes to the setup. Displacing hubs by up to 3 km off the grid changes each policy's mean by less than two points. Increasing the fleet from five to twelve aircraft per hub narrows the replanner's deficit to about two points at 16 and 64 hubs, without reversing the ranking.

Figure 1 shows the mechanism on one surge: the surge is served about equally well, but a hub that gave aircraft later misses its own requests. Figure 2 shows that pattern over whole days. Under the relocating policy at 16 hubs, giving hubs have 154 more late or undelivered requests per day than under local rules; receiving hubs recover seven, leaving 147 more across the region. At 64 hubs, giving hubs lose 16 additional requests per hundred of their own, while receiving hubs serve two more per hundred.

Can a longer surge repay the move? On otherwise quiet 16-hub days, we held one five-hub surge for 15, 60, 120, or 240 minutes at the same request rate, using five matched seeds at each length. At 15 minutes, relocating scores 0.87 against local rules' 0.94; lending and return both score about 0.96. At 60 minutes, lending, the replanner, and Return beat local



Fig. 3. Models score below local rules in both roles. Large points show mean score changes from local rules on matched days; small points show each day: three at 16 hubs, two at 64. Orange points are live managers; outlined points are vetoes over replanner orders. Scores and move counts are in the appendix. Luna's day marks use scores rounded to hundredths.

rules on all five seeds; Return scores 0.73 and the replanner 0.69 against local rules' 0.65, while relocating without lending still trails at 0.63. At 120 minutes all four alternatives beat local rules on all five seeds; at 240, none has a higher mean. Moving can pay in the one- and two-hour surges on this quiet day, especially when aircraft return. The main comparison has 15-minute surges, where the move costs more than it gains. The per-seed scores are in the appendix.

B. A model relocates in every role

Figure 3 puts the four models in the live-manager and veto roles on the same seeded days. Local rules average 0.74 at 16 hubs and 0.71 at 64; the replanner averages 0.71 and 0.62, respectively. Per-seed results are in the appendix. Of the 40 runs, 39 score below local rules. Shown only hubs with a problem, Haiku as a live manager makes 7 to 10 moves per 16-hub day; Sonnet makes 36 to 56, and Opus 54 to 85. GPT-6 Luna makes 1 to 6, yet also trails local rules: it grants 164 to 193 lending permissions instead. At 64 hubs all four models score below local rules' 0.71. Sonnet and Opus make hundreds of moves; Luna grants 1,096 and 1,000 lending permissions on its two days. Live managers can also lose aircraft to exhausted endurance away from a hub, up to 56 of 320 at 64 hubs. Opus beats local rules on one 16-hub seed despite making 85 moves, but trails on the other four tested days. Few moves alone do not make a good fleet plan either: Luna's extensive lending still loses.

As a veto the models are steadier, but still trail local rules on every seed. At 16 hubs, they keep 16 to 34 of the replanner's move orders while removing many of its lending permissions. They do not consistently distinguish a useful temporary permission from a harmful move. No veto run loses an aircraft. The appendix compares Luna's API results with its earlier command-line runs on the same seeds.

Figure 2 retains the original 40 CLI runs, replayed from saved replies and compared with local rules on the same day. For Haiku, Sonnet, and Opus in both roles, and for Luna as a veto, at least 90 percent of the additional lost requests are at hubs that gave aircraft or stock. Luna's CLI live manager is the exception, with only 15 to 20 percent of its loss at giving hubs. In the API reruns, giving hubs account for 25 percent of Luna's additional live-manager losses at 16 hubs and 69 percent at 64. Its veto losses still concentrate at giving hubs. Luna's spatial pattern therefore changes with the client path, especially at 64 hubs.

A model can also write the policy that runs the day. We gave Haiku and Sonnet the mission description and the policy language, ran what they wrote with no further calls, then showed each model two day summaries and asked for one revision. Both policies relocate aircraft and transfer stock on every surge, making about 65 moves a day against the replanner's 25. They score 0.67 (Haiku) and 0.66 (Sonnet) against local rules' 0.74 on three matched 16-hub days. Sonnet's revision halves its moves without improving its score; neither revision reaches local rules.

The behavior does not depend on a busy day. With every event kind at a quarter of its rate at 16 hubs, local rules score 0.92 and 0.94 on two seeds. Haiku and Sonnet still make 17 to 38 moves per day and score 0.88 to 0.93. Almost every scheduled call finds a problem somewhere in the region, and the models act on it.

C. What stops a model from relocating

We tested Haiku and Sonnet as live managers on two matched 16-hub days, changing the view and the checks one at a time. With a summary of every hub, Haiku makes 44 to 52 moves and Sonnet 37 to 66. Reporting the outcome of the last orders reduces Haiku's moves on one seed, but Sonnet moves more on both. A feasibility checker and a line showing what the giving hub kept do not consistently reduce moves either. Information about past costs does not by itself stop the next move.

A reserve rule does. It refuses orders that would leave a hub with too few aircraft or too little stock. With this guard, Haiku makes 10 to 25 moves and Sonnet 28 to 38; 29 to 38 orders are refused per day. The same guard reduces the replanner to 20 or 21 moves, with no model involved. Neither guarded model consistently beats local rules.

Showing only hubs with problems reduces Haiku to seven moves on each 16-hub day without improving its score; Sonnet still makes 36 and 48. This smaller view is the setting of Figure 3. At 64 hubs, all tested models still relocate; there are dozens of problem hubs at each call. A smaller view can change behavior; it does not enforce a reserve.

V. The Fleet API at Scale

In Ripple, a dispatcher assigns requests to aircraft and checks which fleet orders can be carried out. A fleet manager makes the higher-level choices. The manager can be a fixed rule or a language model. The fleet API connects them: it is

TABLE II
WEIGHTED SHARE OF REQUESTS DELIVERED ON TIME. FOR EACH FLEET SIZE, EVERY RULE RAN ON THE SAME NEW SIMULATED DAYS. LOCAL USES NO REGIONAL HELP.

Hubs	Days	Local	Random	Nearest	Spare
16	30	0.738	0.810	0.824	0.843
64	30	0.745	0.809	0.823	0.848
256	15	0.740	0.806	0.822	0.846

the report sent to the manager and the orders the dispatcher will accept. We can change either part and compare the flights that follow on the same mission.

Here the manager has one job. When a hub faces a surge, the dispatcher lists other hubs that can lend aircraft while keeping half of each aircraft class at home. For each hub asking for help, the manager picks one lender from the list or declines help. A selected lender may serve that hub until the surge ends; its aircraft return home after each flight. The dispatcher assigns the individual requests under fixed local rules. This manager cannot relocate aircraft or transfer stock.

We compare two reports for that same decision. The compact report names each asking hub and up to five possible lenders. For each lender it shows distance, aircraft available by type, how many of the asking hub's sites they can reach, the lender's own waiting requests, and time to the first site. The full report adds the status of every hub in the region. We also remove parts of the offer lines one at a time to see which information changes on-time delivery. The allowed help order stays the same in every comparison.

For this API study, we use a separate mission based on a 7 km hub grid and unequal aircraft fleets. Overlapping trauma surges lasting an hour can make the nearest lender different from the one with the most aircraft to spare. At 64 and 256 hubs, we repeat the 16-hub pattern in four and sixteen rotated or mirrored blocks. These larger missions test more simultaneous lending choices of the same kind. We compare local rules with three ways to choose from the same offers. Random chooses any listed lender; nearest chooses the closest; spare chooses the lender with the most available aircraft after subtracting its own waiting requests. The score is the weighted share of requests delivered before their deadlines.

The choice of lender changes the outcome. Lending improves deliveries even when the manager picks from the list at random, but choosing the hub with the most aircraft to spare does better than choosing the nearest or picking at random. This holds from 16 to 256 hubs. Every hub on the list can already spare aircraft and reach a site in time; the difference comes from which one the manager selects.

We then put Claude Haiku and Opus in the manager's place. With the compact report, both score about as well as the spare rule at 16, 64, and 256 hubs. At 16 hubs, the full report also gives no clear gain. On four 64-hub days, it was nearly four times longer than the compact report without a clear gain in deliveries. We did not compare the two reports at 256 hubs, where the compact report itself totals about 155,000 characters over a day.

We also removed facts from the compact report to see whether the models used them. On the 16-hub days, removing the phrase about available aircraft and the sites they can reach lowers both models' scores by about one to two points. Removing the site count alone or the lender's waiting requests produces no clear loss. The useful phrase combines aircraft counts, types, and reach, so these runs do not show which part is responsible.

The models can read the report and choose good lenders across the region. But a rule that counts spare aircraft makes equally good choices at every tested size. For this fleet decision, model calls do not improve on-time delivery.

VI. DISCUSSION AND FUTURE WORK

We described Ripple, a simulator for studying fleet decisions over complete missions. Managers, dispatchers, vehicle behaviors, and motion models can be varied independently on matched days of requests and disruptions. One worker simulated an 8.5-hour day with 5,120 aircraft about 3.4 times faster than real time, without model calls. In the delivery study, language-model managers over-helped, leaving other hubs short later. In the fleet API study, models matched a simple lending rule without improving on it. Ripple shows the effect of each decision on the work completed across the fleet. Those results raise a question about the fleet's design: which decisions should reach the language model?

At the robot level, $\pi_{0.5}$ and GR00T 2 separate choosing a task from carrying it out; GR00T 2 calls the reasoning and action layers System 2 and System 1 [1, 2]. A fleet must make that division across many robots. Flight checks, reserve limits, dispatch, and comparisons between known offers have clear inputs and objectives. Software can make those decisions quickly and consistently. The case for a language model begins when the fleet must interpret a new instruction, resolve competing priorities, or decide when its usual rules no longer fit the task. Giving a model every decision because it sits above the robots is not enough. In our studies, that freedom led to costly help, while a narrower choice gave the model nothing a simple rule could not do.

Ripple can be used to find this boundary. For each proposed model decision, give a rule or planner the same information and authority, then compare completed work over a whole mission. When a rule matches the model, leave that decision with the rule for the task at hand. When a model handles changing goals better, give it that responsibility while the dispatcher keeps flight checks and reserves. The boundary may shift with the mission: relocation helped under longer surges in our study, even though it hurt under the main workload.

These experiments use simulated demand, grid hubs, and kinematic aircraft. The model comparisons cover a small number of days, and the lending study gives the manager a choice with explicit numerical facts.

Future studies should give the manager a different job: interpret new goals, resolve changing priorities, or coordinate robots with different skills. They can also add intermediate managers between a language model and the vehicles, or give a larger model more time to reason. The aim is to learn which decisions benefit from that reasoning and which should stay with rules, planners, and local robot policies.

Generative AI Disclosure

Language models evaluated in the study are named in the experimental setup. An AI assistant helped write code and prose, which the author reviewed.

References

[1] K. Black, N. Brown, J. Darpinian, K. Dhabalia, D. Driess, A. Esmail, M. R. Equi, C. Finn, N. Fusai, M. Y. Galliker *et al.*, "$\pi_{0.5}$: a vision-language-action model with open-world generalization," in *Conference on Robot Learning (CoRL)*, 2025.

[2] NVIDIA, "GR00T 2: Scaling robot foundation models for generalization, dexterity, and adaptation," NVIDIA, Tech. Rep., 2026. [Online]. Available: https://research.nvidia.com/labs/gear/gr00t-2/assets/gr00t2_whitepaper.pdf

[3] S. S. Kannan *et al.*, "SMART-LLM: Smart multi-agent robot task planning using large language models," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2024.

[4] K. Liu, Z. Tang, D. Wang, Z. Wang, X. Li, and B. Zhao, "COHERENT: Collaboration of heterogeneous multi-robot system with large language models," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2025.

[5] Y. Chen, J. Arkin, Y. Zhang, N. Roy, and C. Fan, "Scalable multi-robot collaboration with large language models: Centralized or decentralized systems?" in *IEEE International Conference on Robotics and Automation (ICRA)*, 2024, pp. 4311–4317.

[6] F. Vatnsdal, R. Gopal, R. G. Camargo, V. Kumar, and A. Ribeiro, "Controlling collectives of AI agents in reasoning space with spatial transformers," *arXiv preprint arXiv:2609.28247*, 2026. [Online]. Available: https://arxiv.org/abs/2609.28247

[7] T. Lyu, S. Feng, H. Liu, and H. Yang, "LLM-ODDR: A large language model framework for joint order dispatching and driver repositioning," *arXiv preprint arXiv:2505.22695*, 2025. [Online]. Available: https://arxiv.org/abs/2505.22695

[8] P. Gupta, S. Bae, J. Li, and D. Isele, "Scale-Plan: Scalable Language-Enabled task planning for heterogeneous Multi-Robot teams," *arXiv preprint arXiv:2603.08814*, 2026. [Online]. Available: https://arxiv.org/abs/2603.08814

[9] J. Ludwig, D. Tuohy, and F. Finnigan, "Collaborative LLM-based agents for autonomous multi-UAV mission execution," in *IEEE Aerospace Conference*, 2026.

[10] A. Iannoli, L. Gigli, L. Sciullo, A. Trotta, and M. Di Felice, "Say the mission, execute the swarm: Agent-enhanced LLM reasoning in the web-of-drones," in *IEEE International Symposium on a World of Wireless, Mobile and Multimedia Networks (WoWMoM)*, 2026, pp. 139–148.

[11] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press, "SWE-agent: Agent-Computer interfaces enable automated software engineering," *arXiv preprint arXiv:2405.15793*, 2024. [Online]. Available: https://arxiv.org/abs/2405.15793

[12] C.-Y. Hsieh, S.-A. Chen, C.-L. Li, Y. Fujii, A. Ratner, C.-Y. Lee, R. Krishna, and T. Pfister, "Tool documentation enables zero-shot tool-usage with large language models," 2023. [Online]. Available: https://arxiv.org/abs/2308.00675

[13] A. Lodha, M. P. Varnosfaderani, A. Chakraborty, and A. Mithal, "Less context, better agents: Efficient context engineering for Long-Horizon Tool-Using LLM agents," *arXiv preprint arXiv:2606.10209*, 2026. [Online]. Available: https://arxiv.org/abs/2606.10209

[14] M. Ahn, A. Brohan, N. Brown, Y. Chebotar, O. Cortes, B. David, C. Finn *et al.*, "Do as i can, not as i say: Grounding language in robotic affordances," in *Conference on Robot Learning (CoRL)*, 2022. [Online]. Available: https://arxiv.org/abs/2204.01691

[15] Z. Yang, S. S. Raman, A. Shah, and S. Tellex, "Plug in the safety chip: Enforcing constraints for LLM-driven robot agents," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2024. [Online]. Available: https://arxiv.org/abs/2309.09919

[16] Z. Ravichandran, A. Robey, V. Kumar, G. J. Pappas, and H. Hassani, "Safety guardrails for LLM-Enabled robots," *arXiv preprint arXiv:2503.07885*, 2025. [Online]. Available: https://arxiv.org/abs/2503.07885

[17] S. Kambhampati *et al.*, "Position: LLMs can't plan, but can help planning in LLM-Modulo frameworks," in *International Conference on Machine Learning (ICML)*, ser. PMLR, vol. 235, 2024, pp. 22 895–22 907.

[18] K. DeMarco, E. Squires, M. Day, and C. Pippin, "Simulating collaborative robots in a massive multi-agent game environment (SCRIMMAGE)," in *International Symposium on Distributed Autonomous Robotic Systems*, 2018. [Online]. Available: https://www.kevindemarco.com/assets/pdf/scrimmage.pdf

[19] Zipline, "Zipline fact sheet," Jul. 2023, accessed November 7, 2026. [Online]. Available: https://www.zipline.com/about/zipline-fact-sheet